

LC 1687: Delivering Boxes from Storage to Ports

NUTANIX

© Authored by: Arpit Raj, LNMIIIT Jaipur | August 28, 2026

1. Problem Overview & Basics

- `boxes[i]` → `portsi`, `weighti`
- **Given:** `ports count`, `maxboxes` (mb), and `maxweight` (mw)
(This represents the limit of the ship)
- **Constraint:** Boxes need to be delivered in the order they are given.
- **Output (O/P):** Minimum trips ship needs to make. (Return trip to storage is also counted)

Example Walkthrough:

`Boxes = [[1,1], [2,1], [1,1]]`, `mb = 3`, `mw = 3`

We can take all 3 boxes as they are under the limit.

Path: [Storage] → (Port 1) → (Port 2) → (Port 1) → [Storage]

So, 3 outward trips + 1 return trip = **4 answer**

2. Logic & Sliding Window Approach

- 1 trip is from **storage** → **first port**
- 1 trip is from **last port** → **storage**
- And then additional **x** number of trips.

Where **x** = no. of times port changes b/w **l** and **r**.

Hence, keep a **sliding window** to maintain a valid window.

Constraints:

no of box $\leq$ mb

tot weight $\leq$ mw

"constraints ki vajah se sliding window use karna padega"

Example of Port Changes:

Ports array: [1, 1, 2, 2, 3] → **2 changes** (from 1→2 and 2→3)

3. DP State & Precomputations

- $dp[i]$ → min no. of trips needed to deliver first i boxes.

"total weight ke liye prefix, aur number of changes bhi precompute karke rakh lenge"

Sub-array split: [0 l-1] [l r]

$$\text{Cost} = 2 + \text{cpref}[r+1] - \text{cpref}[l+1]$$

DP Transition:

$$dp[r+1] = \min(dp[r+1], dp[l-1+1] + \text{Cost})$$

4. Optimized C++ Implementation

Note: Sliding window applied to evaluate optimal states greedily.

```
class Solution {
public:
    int boxDelivering(vector<vector<int>>& boxes, int portsCount, int mb,
int mw) {

        int n=boxes.size();
        //dp[i] mincost to deliver first i box;
        //basically 1 go +1 return +number of changes port undergoes
        vector<int> dp(n+1,INT_MAX);
        dp[0]=0;

        vector<int> changepref(n+1,0);

        for(int i=0;i<n;i++){
            if(i>0 && boxes[i][0]!=boxes[i-1][0]){
                changepref[i+1]=changepref[i]+1;
            }
            else{
                changepref[i+1]=changepref[i];
            }
        }

        //since restrictions he
        //sliding window use krenhge to maintain a valid window

        vector<long long> pref(n+1,0);
        for(int i=0;i<n;i++){
            pref[i+1]=pref[i]+boxes[i][1];
        }

        int l=0;
        for(int r=0;r<n;r++){

            while(r-l+1>mb || pref[r+1]-pref[l]>mw){
                l++;
            }

            //writing this next condition took hour of brainstorming as i
            was getting wrong ans
            //on few testcase

            //reason being maybe starting l , might not be optimal
```

```

        //and need to choose greedily a better l ahead of starting l
        //without this condition we were only considering starting l

        //dp[r+1] = dp[l] + 2 + changepref[r+1] - changepref[l+1]
        //rearrange dp[r+1]-2=dp[l]-changepref[l+1]
        //to make dp[r+1] small , make dp[l]-changepref[l+1] small
        //r fixed l is not

        //cost(l)=dp[l]-changepref[l+1];
        //cost(l+1)=dp[l+1]-changepref[l+2]

        while(l<r && dp[l]-changepref[l+1]>=dp[l+1]-changepref[l+2]){
            l++;
        }

        int cost=2+changepref[r+1]-changepref[l+1];
        //[0.....l-1][l.....r]

        dp[r+1]=min(dp[r+1],dp[l-1+1]+cost);
        //r+1 and l-1+1 because of 0 se l-1 me l-1+1 boxes he
    }

    return dp[n];

}

};

```